

**POST GRADUATE DIPLOMA IN IT INFRASTRUCTURE ,
SYSTEMS AND SECURITY
(PG - DITISS)**

Sunbeam , Pune, Maharashtra

**A PROJECT REPORT ON
DevinfraX - Automated Hybrid Infrastructure**

**SUBMITTED TOWARDS
PG-DITISS August 2025**

BY

Group Number – 03

Atharva Chaudhari	PRN: 250844223007
Atharva Deo	PRN: 250844223010
Rushikesh Khot	PRN: 250844223037
Saurabh Tekale	PRN: 250844223052

Under the Guidance of

Mr. Vishal Salunke	Mr. Gajanan Taur
Course Co-Ordinator	Project Guide

Contents

Abstract	2
1 Introduction	3
2 Objectives	4
3 Hybrid Infrastructure Overview	5
Scope	5
4 System Architecture and Node Responsibilities	6
5 CI/CD Workflow and Deployment Flow	8
6 Tools and Technologies Used	10
7 Results	11
8 Advantages	14
9 Future Enhancements	15

Abstract

In today's fast-paced software development environment, automation, reliability, and scalability are essential for delivering high-quality applications. DevinfraX – Automated Hybrid Infrastructure is a DevOps-oriented project that focuses on implementing a fully automated Continuous Integration and Continuous Deployment (CI/CD) pipeline using a hybrid infrastructure model.

The project integrates industry-standard tools such as Jenkins, Ansible, Docker, SonarQube, Trivy, JMeter, Prometheus, and Grafana to automate the complete software delivery life-cycle—from code integration to production monitoring. The infrastructure consists of cloud-based AWS EC2 instances and on-premise virtual machines, enabling realistic deployment across multiple environments including Stage, Pre-Production, and Production.

DevinfraX ensures that only quality-tested, secure, and performance-validated applications reach production by enforcing multiple validation gates. The system also provides continuous monitoring of production environments to ensure application availability and system health. This project demonstrates real-world DevOps practices and serves as a scalable, reliable, and industry-relevant solution for automated application deployment.

Chapter 1

Introduction

In modern software development, rapid delivery, reliability, and scalability are critical. Manual deployment processes are error-prone, time-consuming, and difficult to scale. DevOps practices address these challenges by automating the software delivery lifecycle.

DevinfraX – Automated Hybrid Infrastructure is a DevOps-based project that demonstrates an end-to-end CI/CD pipeline deployed on a hybrid infrastructure. The project integrates multiple industry-standard tools to automate code integration, containerization, testing, security scanning, performance validation, deployment, and monitoring.

The primary objective of DevinfraX is to ensure that only high-quality, secure, and performance-tested applications reach production, with minimal human intervention.

Chapter 2

Objectives

The key objectives of DevinfraX are:

- To automate the complete application deployment lifecycle
- To implement CI/CD using Jenkins
- To containerize applications using Docker
- To enforce code quality and security checks
- To perform performance testing before production deployment
- To deploy applications across a hybrid infrastructure
- To monitor the production environment continuously

Chapter 3

Hybrid Infrastructure Overview

DevinfraX is implemented on a hybrid infrastructure, combining cloud-based and on-premise virtual environments. The infrastructure consists of four machines, each assigned a specific role in the CI/CD pipeline.

Infrastructure Components

Cloud Environment(AWS EC2)

- Control Node
- Production Node

On-Premise / Virtual Environment

- Stage Node
- Pre-Production Node

This hybrid approach reflects real-world enterprise environments where workloads are distributed across cloud and local systems.

Chapter 4

System Architecture and Node Responsibilities

Control Node (EC2 Instance)

The Control Node acts as the central orchestrator of the entire system. The Control Node ensures automation and coordination across all environments.

Installed Tools:

1. Jenkins
2. Ansible

Responsibilities:

1. Triggering CI/CD pipelines upon code changes
2. Building the application
3. Creating Docker images
4. Pushing images to Docker Hub
5. Orchestrating deployments across Stage, Pre-Prod, and Prod nodes using Ansible

Stage Node

The Stage Node is the first validation environment in the pipeline. This stage acts as a quality and security gate, preventing defective or insecure code from progressing further.

Installed Tools:

1. Docker
2. SonarQube
3. Trivy

Responsibilities:

1. Running the application container
2. Performing code quality analysis using SonarQube
3. Scanning Docker images for vulnerabilities using Trivy

Pre-Production Node

The Pre-Production Node closely resembles the production environment. This stage ensures that the application is stable and scalable before deployment to production.

Installed Tools:

1. Docker
2. jmeter

Responsibilities:

1. Running the same Docker image used in other stages
2. Conducting performance and load testing
3. Validating application behavior under stress

Production Node (EC2 Instance)

The Production Node hosts the live application accessed by end users. Monitoring ensures high availability and proactive issue detection.

Installed Tools:

1. Docker
2. Prometheus
3. Grafana

Responsibilities:

1. Running the production application container
2. Collecting system and application metrics
3. Visualizing performance and health data through dashboards

Chapter 5

CI/CD Workflow and Deployment Flow

The DevinfraX pipeline follows a structured and automated flow.

Step 1 : Code Integration

1. Developers push application code to GitHub.
2. GitHub acts as the single source of truth.

Step 2 : Continuous Integration

1. Jenkins pipeline is triggered automatically.
2. The application is built.
3. A Docker image is created.
4. The image is pushed to Docker Hub.

Step 3 : Stage Environment Validation

1. The Docker image is deployed on the Stage VM.
2. SonarQube performs code quality checks.
3. Trivy scans the image for vulnerabilities.
4. Pipeline proceeds only if all checks pass.

Step 4 : Pre-Production Testing

1. The same Docker image is deployed on the Pre-Prod VM.
2. JMeter performs load and performance testing.

3. Performance thresholds are validated.

Step 5 : Production Deployment

1. After successful validation, the image is deployed to the Prod EC2.
2. The application becomes accessible to end users.

Step 6 : Monitoring and Observability

1. Prometheus continuously collects metrics.
2. Grafana visualizes metrics through dashboards.
3. Alerts can be configured for proactive monitoring.

Chapter 6

Tools and Technologies Used

1. **GitHub** – Version control and source code management
2. **Jenkins** – CI/CD pipeline automation
3. **Ansible** – Configuration management and orchestration
4. **Docker** – Application containerization
5. **Docker Hub** - Container image registry
6. **SonarQube** - Static code quality analysis
7. **Trivy** - Vulnerability scanning
8. **JMeter** - Performance and load testing
9. **Prometheus** - Metrics Collection
10. **Grafana** - Monitoring and Visualization
11. **AWS EC2** - Cloud Infrastructure

Chapter 7

Results

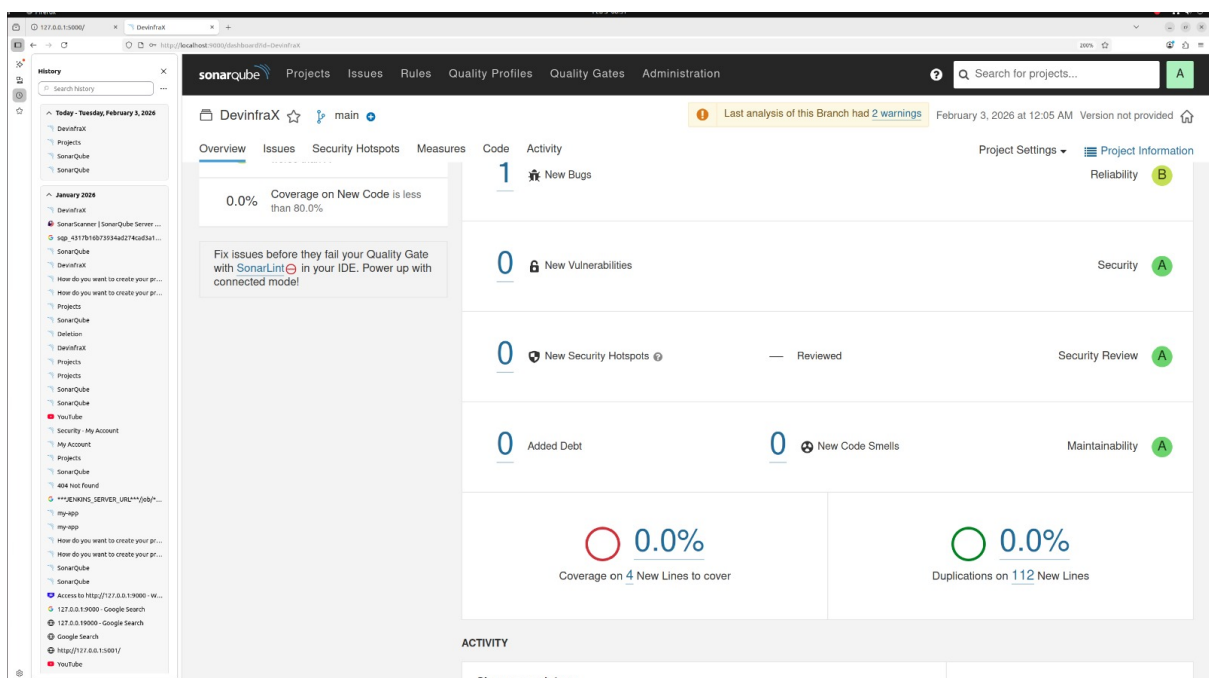


Figure 7.1: SonarQube on 9000



Figure 7.2: Trivy

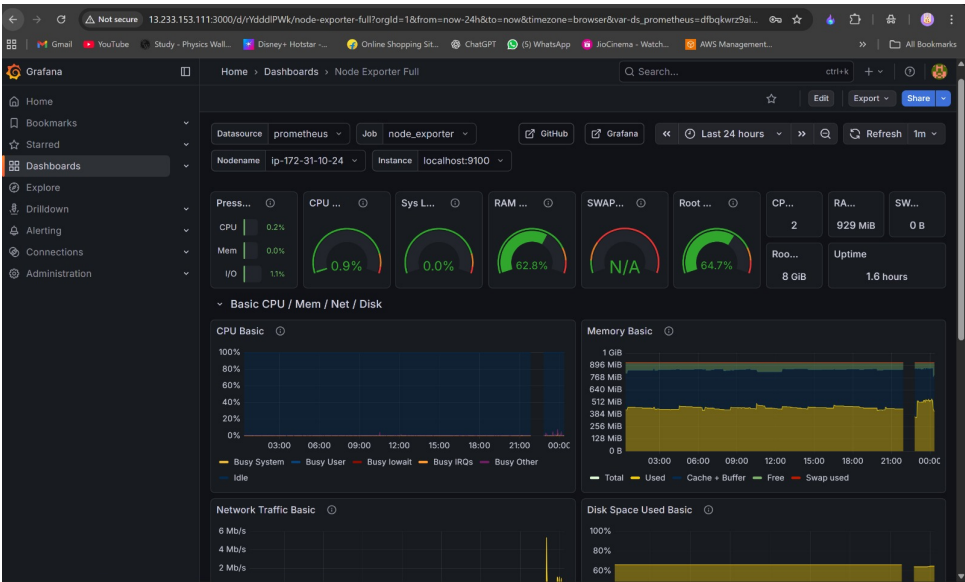


Figure 7.3: Monitoring

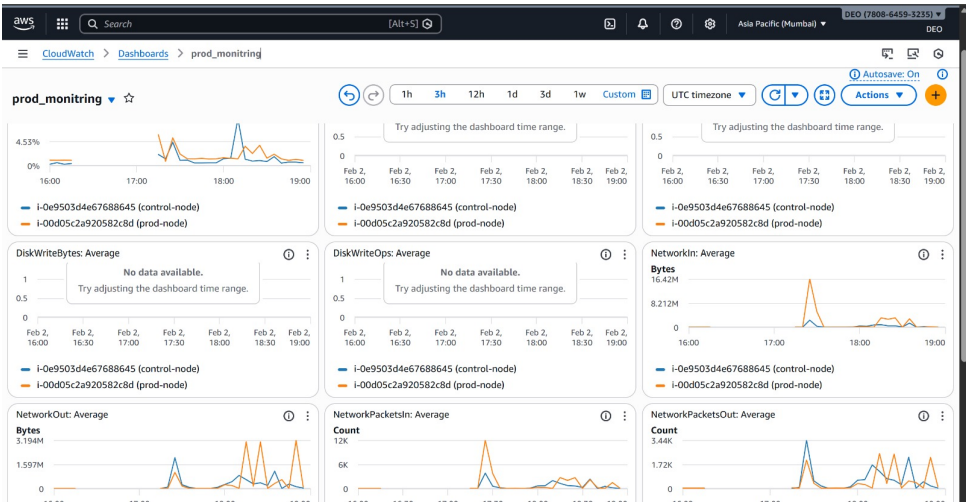


Figure 7.4: CloudWatch UI

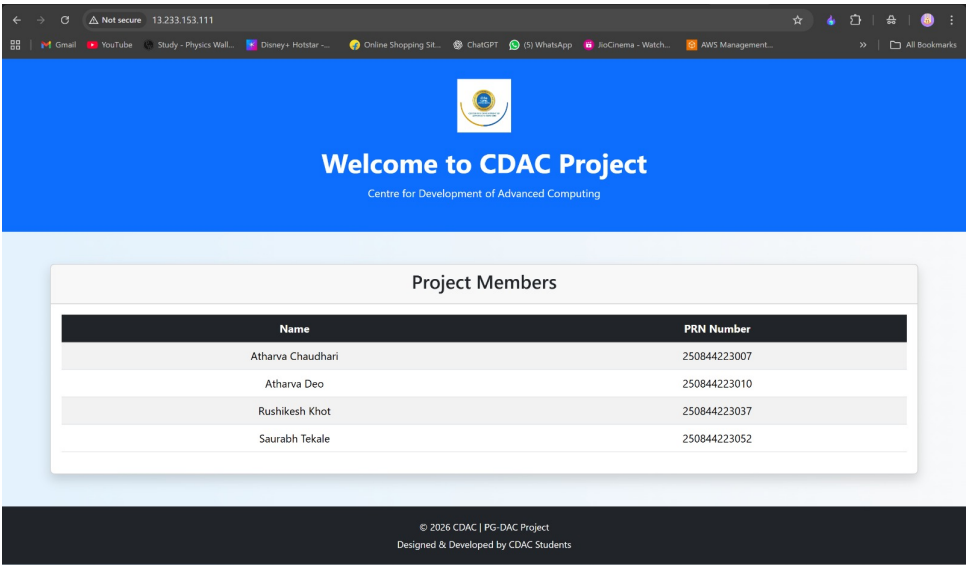


Figure 7.5: Final UI Image

Chapter 8

Advantages

1. Fully automated deployment pipeline
2. Consistent environment using Docker images
3. Early detection of code and security issues
4. Performance validation before production
5. Hybrid infrastructure implementation
6. Continuous monitoring and observability

Chapter 9

Future Enhancements

1. Separate monitoring infrastructure for scalability
2. Auto-scaling using Kubernetes
3. Centralized logging using ELK Stack
4. Automated rollback strategies
5. Cloud-native CI/CD enhancements

Conclusion

DevinfraX – Automated Hybrid Infrastructure is a comprehensive DevOps project that showcases the practical implementation of CI/CD pipelines, containerization, testing, deployment, and monitoring across a hybrid environment.

By integrating Jenkins, Ansible, Docker, and monitoring tools, the project delivers a robust, scalable, and production-ready solution. It highlights the importance of automation, quality control, security, performance testing, and monitoring in modern software delivery.